System Verification and Validation Plan for Keystone

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

November 1, 2026

Revision History

Date	Version	Notes
2026-10-29	1.0	General Information
2026-10-31	1.1	Plan and the SRS validation walkthrough appendix

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 3

3 Plan **3**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 4
- 3.3 Design Verification . . . 4
- 3.4 Verification and Validation Plan Verification . . . 6
- 3.5 Implementation Verification . . . 6
- 3.6 Automated Testing and Verification Tools . . . 7
- 3.7 Software Validation . . . 8

4 System Tests **9**
- 4.1 Tests for Functional Requirements . . . 9
  - 4.1.1 Area of Testing1 . . . 9
  - 4.1.2 Area of Testing2 . . . 9
- 4.2 Tests for Nonfunctional Requirements . . . 10
  - 4.2.1 Usability . . . 10
  - 4.2.2 Installability . . . 10
  - 4.2.3 Portability . . . 10
- 4.3 Traceability Between Test Cases and Requirements . . . 10

5 Unit Test Description **10**
- 5.1 Unit Testing Scope . . . 10
- 5.2 Tests for Functional Requirements . . . 10
  - 5.2.1 Module 1 . . . 10
  - 5.2.2 Module 2 . . . 11
- 5.3 Tests for Nonfunctional Requirements . . . 11
  - 5.3.1 Module ? . . . 11
  - 5.3.2 Module ? . . . 12
- 5.4 Traceability Between Test Cases and Modules . . . 12

6 Appendix **13**
- 6.1 Symbolic Parameters . . . 13
- 6.2 Usability Survey Questions? . . . 13
- 6.3 SRS Validation Walkthrough and Checklist . . . 13
  - 6.3.1 Agenda . . . 13
  - 6.3.2 Questions for each use case . . . 13
  - 6.3.3 Checklist . . . 14

7 Appendix — Generative AI Use Disclosure **17**
7.1 AI Tools Used . . . 17
7.2 How and Where Used . . . 17
7.3 Verification of AI-Generated Content . . . 17

8 Appendix — Reflection **18**

List of Tables

1 Verification and validation roles . . . 5
2 Automated testing and verification tools . . . 8
3 Agenda of the SRS walkthrough with Dr. Smith . . . 13
4 SRS validation checklist . . . 14

List of Figures

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

This document describes how Keystone will be verified (are we building the product right?) and validated (are we building the right product?). Section 2 gives general information, including what is tested, what is not, and which documents the plan relies on. Section 3 describes the plan: who verifies what, how the SRS, design, this plan, and the implementation are verified, which tools are used, and how the software is validated with stakeholders. Section 4 describes the system tests, and Section 5 describes the unit tests. The appendix contains the material used to validate the SRS with the course instructors.

2 General Information

2.1 Summary

The software under test is Keystone, a web-based capstone project management system for the Software Engineering (SE), Computer Science (CS), and Mechatronics (TRON) capstone courses. Keystone covers the lifecycle from project proposal to team project approval. Its main functions are:

- structured project proposals by supervisors and student teams, with instructor review before a project is listed;
- a searchable project catalog showing project status and supervisor capacity;
- student and team profiles, team formation under course-specific rules, and teammate and project recommendations;
- team applications to projects, supervisor and team acceptance, and comment threads attached to projects and applications;
- team project submission with automated checks, and approval by instructors and TAs, either in the platform or recorded from an external process;
- course enrollment import from Avenue to Learn or file upload, validated team composition export, and email notifications and checklists;
- role-based access control, authentication through McMaster SSO or another trusted provider, and audit logging.

The requirements being verified are listed in the SRS (Patel et al., 2026f).

2.2 Objectives

The objectives are ordered by priority, based on the risks identified in the SRS: errors in the approval workflow affect course records, errors in access control expose student data, and a hard-to-use platform will not be adopted by supervisors.

In scope.

1. **Functional correctness.** Build confidence that the functional requirements (`FR_SupervisorProposal` to `FR_AssignTA` in the SRS) are met, especially the project and team-project status lifecycle, supervisor capacity (slot reservation and release), team formation rules, submission validation, enrollment import, and external supervisor verification and scoping.

2. **Access control and privacy.** Demonstrate that users can only see and change what their role and course permit, and that data is protected in transit and at rest (`NFR_Authenticate`, `NFR_Authorize`, `NFR_EncryptTransit`, `NFR_EncryptRest`, `DPR_RoleBasedDataPermissions`, `DPR_VisibilityMatrix`, `DPR_EncryptionAtRest`).

3. **Usability.** Demonstrate that first-time users in the main roles (students, TAs, and professors) can complete their primary tasks within the usability time limit, and find the platform intuitive and easy to use (`NFR_UsabilityFirstSuccess`, `NFR_LearnUnassisted`).

4. **Performance and installability.** Demonstrate that the system meets its latency target under the specified load, and that an IT professional can deploy it from the deployment guide within the time limit (`NFR_PerfTail`, `NFR_PeakUsers`, `NFR_DeployUnassisted`).

5. **Portability and accessibility.** Demonstrate that the system works on the target browsers and in Docker, and passes an automated accessibility audit (`NFR_Browsers`, `NFR_Container`, `NFR_AccessibilityWCAG`).

Out of scope.

- **Quality of recommendations.** We will test that recommendations are produced and displayed (`FR_TeamRecommend`), but not whether they are good matches. Recommendation quality is a stretch goal in the SRS, and judging it would need a study with real students.
- **External systems.** We assume that McMaster SSO, Avenue to Learn, the email service, and third-party libraries have been verified by their developers. We test our handling of their failures (for example, an unavailable Avenue), not their correct operation. Following the SRS Development Plan, email delivery and authentication may be simulated during the course.
- **Availability over time.** The availability criterion is measured over a long window (see the SRS), which cannot be observed within the course timeline. We will verify that the health check works and that the system meets the target over the period that the demo allows.
- **Maintainability.** We will review the likely changes in the SRS against the design, but will not measure the actual effort of making changes.
- **Features not implemented.** Requirements that the SRS Development Plan marks as future work, or as Phase 3 and not completed, will not be tested.
- **Formal verification.** We do not plan formal proofs.
- **Industry sponsors.** The usability test does not include external supervisors from industry, who have no McMaster account and are the least familiar with the course. Professors in the test represent internal supervisors and instructors. Findings about external supervisors will need follow-up testing if one becomes available.

2.3 Custom Documents

This project has two Custom Documents: a usability testing report and an interview report.

The **usability testing report** records the results of the usability test described in Section 4.2.1 and supports the Usability NFR in the SRS. In the test, we observe TAs, students, and professors using Keystone to complete the primary tasks of their roles, to judge whether the platform is

intuitive and easy to use without training. The report covers the participants and their roles, the tasks given, completion times, problems and points of confusion observed, and the changes made as a result.

The **interview report** records the stakeholder interviews that the requirements were derived from: students, external and internal supervisors, and the SE, TRON, and CS course instructors. It contains the interview notes, the requirements and design decisions each note led to, and the stakeholder feedback on the SRS and on the working system. It supports validation by checking that Keystone solves the problems stakeholders described, and the SRS (Appendix A) summarizes it.

2.4 Relevant Documentation

The following documents define what is verified and how the verification is organized.

- **Software Requirements Specification (SRS)** (Patel et al., 2026f). This is the main source for this plan. System tests are derived from its functional requirements, nonfunctional requirements and fit criteria, use cases, and data requirements. The SRS defines the symbolic constants (such as the number of concurrent users and the latency limit) that the nonfunctional tests use, and its traceability matrices will be extended to link each requirement to the tests in this document.
- **Hazard Analysis** (Patel et al., 2026b). It identifies hazards such as failures of the external approval system and access control failures, and the safety and security requirements that follow from them. Tests are added for each of these requirements.
- **Problem Statement and Goals** (Patel et al., 2026e). It records the problems and goals that validation checks against. Validation with stakeholders asks whether the system solves these problems, not only whether it meets the SRS.
- **Development Plan** (Patel et al., 2026a). It defines the tools and workflow (such as version control, continuous integration, and linting) that the automated testing in this plan relies on.
- **Module Guide (MG) and Module Interface Specification (MIS)** (Patel et al., 2026c,d). These will define the modules and interfaces that the unit tests in this plan target. The unit test sections will be completed after the design is finished.

3 Plan

This section describes who verifies and validates Keystone, and how. Section 3.1 assigns roles. Sections 3.2 to 3.5 describe the verification of the SRS, the design, this plan, and the implementation. Section 3.6 lists the automated tools, and Section 3.7 describes how the software is validated with stakeholders.

Handling review feedback. All reviews in this plan follow one process, which is not repeated in each subsection. Every finding from a review, walkthrough, inspection, or stakeholder meeting is recorded as a GitHub issue with the *Verification* label (and the *Documentation* or *Bug* label where relevant), and is assigned to a team member. The assignee fixes the finding on a branch and opens a pull request that references the issue. The pull request goes through the normal pull request review

described in the Development Plan (Patel et al., 2026a), and the issue is closed only after the pull request is merged and the reviewer who raised the finding, or a teammate, confirms that the change addresses it. A finding that the team decides not to act on is closed with a written reason.

3.1 Verification and Validation Team

Verification tasks rotate between team members, and no one verifies work that they wrote alone. The roles are in Table 1. The documentation and technical QA role rotates per deliverable in the order set in the Development Plan (Patel et al., 2026a): Ahmed, Andrew, Ayush, Waqar, and then repeats.

3.2 SRS Verification

The SRS (Patel et al., 2026f) is verified in four steps before it is treated as the basis for design and testing.

1. **Internal review.** Each team member reviews the sections that they did not write, using the SRS checklist from the course. The reviewer also checks the qualities that the SRS template asks for: unambiguous, consistent, complete, validatable, and abstract. The documentation and technical QA member then checks traceability: every stakeholder requirement maps to a use case and a functional requirement, every functional requirement maps to a stakeholder requirement or a nonfunctional requirement, every "Traces To" entry in the Hazard Analysis (Patel et al., 2026b) names a requirement that exists in the SRS, and every requirement has a fit criterion that can be tested.

2. **TA review.** Chris Schankula reviews the SRS and provides feedback.

3. **Peer review.** Classmates review the SRS using the Peer Review issue template, with the SRS checklist and the rubric.

4. **Validation with the instructors.** Reviewing the SRS ourselves cannot show that it matches how the courses really work, so Dr. Smith validates the facts that only he can confirm, through a structured walkthrough and a checklist. These are described below.

Walkthrough and checklist with Dr. Smith. We hold one 45 minute session with Dr. Smith, in the first supervisor meeting after the SRS is submitted and before the design document is started. The team presents instead of asking him to read the SRS, because the SRS is long and the questions that matter are about the use cases and the lifecycle. The agenda is in Table 3. We walk through the project and team-project status lifecycle and four use cases: `UC_ReviewProposal`, `UC_ApproveTeamProject`, `UC_ImportEnrollment`, and `UC_FollowUpUnselected`. These are the ones where the instructor's own process and decisions matter most. For each one, the presenter reads the main success scenario and the alternative flows and asks the targeted questions listed in Appendix 6.3. Dr. Smith then marks a short checklist of the statements in the SRS that only he can confirm, each as correct, needs a change, or unsure. The agenda, the questions, and the checklist are in Appendix 6.3. His answers are recorded in the Supervisor Meeting issue, and every "needs a change" or "unsure" answer becomes a Verification issue as described above.

Role	Members	Description
Documentation and technical QA	Rotates per deliverable: Ahmed, Andrew, Ayush, Waqar	Checks each deliverable against the repository checklist and the Avenue rubric before submission, runs the traceability check between documents, and flags gaps in time to fix them. The same member performs the post-merge functional QA for code in that deliverable period, and owns the system test results for it.
Unit test owner	Ayush Patel	Owns the backend (pytest) and frontend (Jest) unit tests and their coverage, and reviews unit tests in pull requests for whether they test the intended behaviour.
Integration test owner	Ahmed Rashrash	Owns the integration tests, including backend with the PostgreSQL database, and the interactions with the email service, Avenue to Learn, and the authentication providers.
End-to-end test owner	Andrew Habib	Owns the automated end-to-end (Playwright) tests for the most common use cases, including the cross-browser, screen-size, and accessibility checks.
CI/CD and automation owner	Waqar Ul-Hassan	Owns the GitHub Actions pipeline, the static analysis, coverage, and SonarQube setup, the performance test harness, and the mutation testing run, so that all checks run automatically as described in Section 3.6.
Verification issue tracking	Andrew Habib (GitHub Manager)	Keeps the *Verification* label, issue templates, and project board consistent, and checks before each milestone that no review finding is left open or unassigned.
Reviewers	All four team members	Review each other's pull requests and take part in the design and code walkthroughs.
Standing reviewer	Chris Schankula (TA)	Reviews and provides feedback on each deliverable
External reviewers and validators	Dr. Spencer Smith (supervisor and SE instructor), Dr. Mehdi Moradi (CS instructor), Dr. Alan Wassyng (TRON instructor).	Not members of the team. They judge whether the requirements and the software match how their courses work.

Table 1: Verification and validation roles

3.3 Design Verification

The design documents are the Module Guide and the Module Interface Specification (Patel et al., 2026c,d). They are verified as follows.

1. **Checklist review.** Teammates, the TA, and classmates review the documents using the course's MG and MIS checklists and the rubric.

2. **Requirements coverage check.** The documentation and technical QA member builds a table mapping every functional requirement and every safety and security requirement to the module(s) that satisfy it, and every module to the requirement(s) it exists for.

3. **Hazard scenario review.** For each hazard in the FMEA of the Hazard Analysis (Patel et al., 2026b), the team identifies the design element that mitigates it and checks that the mitigation is in the design and not only in the requirements. The review gives extra attention to the places where the design is most likely to fail such as the status lifecycle and slot reservation logic, simultaneous requests (race conditions), and the enforcement of visibility rules on every read path.

4. **Design walkthrough with Dr. Smith.** We walk through the module decomposition with him using scenarios, because modularity is his main design concern. Example scenarios are: an instructor disables a feature, a course uses external instead of in-platform approval, a course changes its team size rules after teams have formed, and a second university uses Keystone with a different enrollment source. For each scenario, we trace which modules change and which do not, and he judges whether the result is acceptable.

3.4 Verification and Validation Plan Verification

This plan is verified in the following ways.

1. **Review.** Teammates, the TA, and classmates review the plan using the course's VnV plan checklist and the rubric.

2. **Coverage check.** We confirm that every in-scope requirement in the SRS has at least one test, using the traceability table in this document. Requirements that carry the most risk (the status lifecycle and slot reservation, team size limits, role-based visibility, and enrollment import) must have more than one test, so that a passing test suite gives real confidence.

3. **Hazard check.** For each hazard in the Hazard Analysis, we ask which test would fail if the hazard occurred. A hazard with no such test is a finding.

4. **Validation review with Dr. Smith.** He reviews only the software validation plan (Section 3.7) and the usability test protocol, because this is where his input is most useful.

5. **Mutation testing (planned extra, high-risk areas only).** To check that the automated tests can detect faults, the team will run mutation testing on the highest-risk backend modules only. This includes the status lifecycle and slot reservation logic, the team formation rule checks, and the role-based visibility checks. The tool is mutmut. The run happens before the Revision 0 demo and before the final demo. Surviving mutants are examined, and each one that shows a missing test becomes a Verification issue. If time runs short, mutation testing is dropped without affecting any other part of this plan.

3.5 Implementation Verification

Implementation verification combines dynamic testing, static analysis, and a code walkthrough.

Dynamic testing

- **Unit tests (owner: Ayush).** Backend unit tests with pytest cover the API endpoints and the core logic. Frontend unit tests with Jest cover components, including form validation and the display of statuses. Test design follows the guidance in the Development Plan (Patel et al., 2026a), including boundary values and expected failures. The target is about 80% statement coverage. Section 5 describes the unit tests in detail.

- **Integration tests (owner: Ahmed).** These run the backend against a real PostgreSQL database in Docker, to test the transactions, constraints, and locking that the safety requirements depend on. The email service is replaced by a local test mail server (for example Mailpit), so tests can check what is sent, to whom, and how often. Avenue to Learn is replaced by a stub that can return a valid list, a malformed list, a duplicate entry, or no response, so that we test our handling of its failures, as the General Information section describes. Authentication providers are simulated in the same way.

- **End-to-end tests (owner: Andrew).** Playwright tests drive the real application in a browser through the most important use cases for each role. They run in Chrome, Firefox, and Safari (WebKit) at the three screen widths named in the SRS, and include an automated accessibility audit (axe-core) of every page.

- **Performance tests (owner: Waqar).** A load test with Locust simulates the number of concurrent users, think time, and 95th percentile latency limit set in the nonfunctional requirement `NFR_PerfTail` of the SRS. The load test runs before milestones, not on every pull request, because it is slow and needs a full deployment. A first baseline run is done once the project catalog and application flow exist, and a run is done before each milestone from the Revision 0 demo onward, so that slow queries are found early.

Static verification

- **Static analyzers and linters.** Ruff and ESLint run on all code, as in the Development Plan (Patel et al., 2026a). We add a static type checker for the backend (mypy) and strict mode in TypeScript, because type errors in status and role handling are a likely source of faults. SonarQube analyzes code quality and coverage reports. Dependency audits (`pip-audit` and `npm audit`) check for known vulnerabilities.

- **Code walkthrough.** Before the Revision 0 demo, the team and the TA hold a walkthrough of the modules where a fault would cause the most harm: the status lifecycle and slot reservation logic, the visibility and role checks (including the external supervisor limits), and the notification queue and enrollment import. The author of a module presents it, and the other participants inspect it against a short checklist: the code follows the module interface specification, multi-step changes happen in one transaction, every read path checks permissions on the server, errors are handled explicitly, course rules are read from configuration and not written in the code, and each branch has a test. The session is recorded in a Team Meeting issue, and findings follow the process above. This is in addition to normal pull request review.

3.6 Automated Testing and Verification Tools

The team follows continuous integration and delivery (CI/CD) practice, using GitHub Actions as described in the Development Plan (Patel et al., 2026a). Table 2 lists the tools, what they are used for, and when they run. Pull requests cannot be merged while a required check fails. Falling below the 80% statement coverage target does not block a merge but is flagged for review, as the Development Plan states.

Tool	**Purpose**	**When it runs**
Ruff, ESLint	Linting and style for Python and TypeScript	Every push and pull request
mypy, TypeScript strict mode	Static type checking	Every push and pull request
pytest, pytest-cov	Backend unit tests and coverage (Cobertura XML)	Every push and pull request
Jest	Frontend unit tests and coverage (LCOV)	Every push and pull request
SonarQube	Code quality analysis, and coverage reporting from the pytest-cov and Jest reports	Every pull request and merge to main
pip-audit, npm audit	Known vulnerabilities in dependencies	Every pull request and merge to main
Docker with PostgreSQL, test mail server (Mailpit), Avenue stub	Integration and concurrency tests	Every pull request and merge to main
Playwright, axe-core	End-to-end tests in Chrome, Firefox, and WebKit, and the accessibility audit	Every pull request and merge to main
Locust	Load test at the load given in `NFR_PerfTail`	Manually started workflow before milestones (not on every pull request)
mutmut (planned extra)	Mutation testing of the high-risk backend modules	Manually started before the Revision 0 and final demos

Table 2: Automated testing and verification tools

3.7 Software Validation

Validation checks that we are building the right product: one that solves the problems in the Problem Statement (Patel et al., 2026e), and not only one that meets the SRS. It is done with stakeholders in the following ways. The details of each will be given later, in the usability test protocol (Section 4.2.1) and the Usability Testing Report.

- **Usability testing.** Students, TAs, and professors complete the primary tasks of their roles, as described in the Custom Documents section.
- **Stakeholder interviews.** We interview students, supervisors, and instructors again after the requirements and the first working version are available, to check that the platform addresses what they told us.
- **Demonstrations.** The Proof of Concept demo, the Revision 0 demo, and the final demo are used as validation points with Dr. Smith. After the Revision 0 demo, we also demonstrate the platform to Dr. Moradi and Dr. Wassyng and students.

As the General Information section states, external supervisors from industry are not part of the usability test.

4 System Tests

4.1 Tests for Functional Requirements

4.1.1 Area of Testing1

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output:

   Test Case Derivation:

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output:

   Test Case Derivation:

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

4.2.1 Usability

..to be completed.

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input/Condition:
   Output/Result:
   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input:
   Output:
   How test will be performed:

4.2.2 Installability

..to be completed.

4.2.3 Portability

..to be completed.

...

4.3 Traceability Between Test Cases and Requirements

5 Unit Test Description

5.1 Unit Testing Scope

5.2 Tests for Functional Requirements

5.2.1 Module 1

1. test-id1

Type:
Initial State:
Input:
Output:
Test Case Derivation:
How test will be performed:

2. test-id2

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

5.3.1 Module ?

1. test-id1

   Type:
   Initial State:
   Input/Condition:
   Output/Result:
   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input:
   Output:
   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

References

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Development plan. 2026a. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Hazard analysis. 2026b. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Module guide. 2026c. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/Design/SoftArchitecture/MG.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Module interface specification. 2026d. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Problem statement and goals. 2026e. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/ProblemStatement/ProblemStatement.pdf.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Software requirements specification. 2026f. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/SRS/SRS.pdf.

6 Appendix

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

6.3 SRS Validation Walkthrough and Checklist

This appendix contains the material used to validate the SRS (Patel et al., 2026f) with Dr. Smith (Section 3.2). It has three parts: the agenda of the walkthrough, the questions asked for each use case, and the checklist that Dr. Smith completes.

6.3.1 Agenda

The session lasts 45 minutes. A team member presents, and a second member records answers in the Supervisor Meeting issue. The checklist and agenda are given to Dr. Smith before the session.

Minutes	Topic	What we show
3	Introduction	Purpose of the session, how answers are recorded, how the checklist works
5	Status lifecycle	The project and team-project status lifecycle diagram and the slot (capacity) rule
8	`UC_ReviewProposal`	Main success scenario and the alternative flows (changes requested, rejected)
12	`UC_ApproveTeamProject`	Main success scenario, the automated checks, and the alternative flows
7	`UC_ImportEnrollment`	Main success scenario, the preview, and the alternative flows (Avenue unavailable, absent student)
5	`UC_FollowUpUnselected`	Main success scenario and the list of unselected projects
5	Remaining checklist items and close	Visibility rules, scope, and priorities

Table 3: Agenda of the SRS walkthrough with Dr. Smith

6.3.2 Questions for each use case

Status lifecycle.

- Are these the statuses that you would want to see for a project and for a team on a project, and are any missing or unnecessary?
- Is it right that a team's application does not use a slot, and that a slot is reserved only when the supervisor accepts?

`UC_ReviewProposal.`

- Is requesting changes and receiving a revised proposal enough, or do you need to edit a proposal yourself?
- Is it acceptable that an external supervisor is verified only when you approve a proposal that they submitted?

`UC_ApproveTeamProject.`

- Which parts of the review must never be automated?
- How should a change to an already approved project be handled in your course?
- In external approval mode, who records the outcome in the platform, and how soon?

`UC_ImportEnrollment.`

- Is the Avenue enrollment list the list that you use, and does it include each student's program?
- How do you handle students who join late or come from other departments today?
- What should happen to a student who is on a team but missing from a new import?

`UC_FollowUpUnselected.`

- Is "closed with no approved team" the right definition of an unselected project?
- What do you want to see on the list (for example project title, supervisor, and contact details)?
- Do you want the platform to send the thank-you message, or only produce the list?

6.3.3 Checklist

For each statement, the reviewer ticks *OK* if it is correct, *Change* if it needs to change (with a note)

Table 4: SRS validation checklist

#	Statement in the SRS	SRS reference	OK	Change
A. Project proposals				
1	A supervisor or student team submits a proposal. Only the instructor decides: list it (status Open), request changes, or reject it. TAs do not take part in proposal approval.	`UC_ReviewProposal`, `FR_ReviewProposal`, `SR_TAApproval`	☐	☐
2	Approving an unverified external supervisor's own proposal verifies them, and nothing else does. A verified supervisor can then see student profiles and student-proposed projects, but never another supervisor's projects.	`FR_ExternalSupervisorVerification`, `FR_ExternalSupervisorScope`	☐	☐

3	The required proposal fields are title, background, objectives, impact, completion status, industry, hardware or non-hardware, supervisor capacity, confidentiality, and IP. The optional fields are requirements, deliverables, proposed solution, and additional terms.	FR_SupervisorProposal, FR_ProposalOptionalFields	☐	☐
B. Selection and agreement				
4	An application does not use a slot. A slot is reserved when the supervisor accepts a team, a project is Full only when all its slots are reserved, and a team is Agreed only when both sides have accepted.	Capacity rule (Section on the lifecycle), UC_SelectProject	☐	☐
5	The project statuses (Draft to Closed) and the team-project statuses (Applied to Declined or withdrawn) are complete and correctly named.	Status lifecycle figure	☐	☐
6	A project is Closed when the selection period ends. A Closed project with no approved team is unselected, and the instructor can list these to follow up with their supervisors.	UC_FollowUpUnselected, FR_ReadCourseInformation	☐	☐
C. Team project approval				
7	Before a submission reaches the instructor, the platform checks that required fields and documents are present, the repository exists and is set up correctly, the instructor is a collaborator, every member is on exactly one team, the team has no other approved project, and the project is available.	FR_ApprovalPreflightValidate, UC_ApproveTeamProject	☐	☐
8	The instructor or a TA approves, rejects, or requests changes. The platform never decides automatically.	SR_HumanApproval, SR_TAApproval, FR_ApprovalResponse	☐	☐
9	A course can approve in the platform, record outcomes by hand from an external process, or (where an adapter exists) synchronize with one. Recording by hand is enough for the course's current process.	FR_ExternalApproval, approval mode definition	☐	☐
10	A change to an approved project stays pending, and the original approval stays in effect until the instructor accepts the change.	FR_ApprovalRevision	☐	☐
D. Enrollment, teams, visibility, and scope				
11	Enrollment is imported from Avenue (or a file) with a preview. Duplicates and malformed entries are flagged, students missing from a new import are flagged and never removed automatically, and late students can be added by hand.	UC_ImportEnrollment, FR_ImportEnrollment, FR_EnrollmentDiscrepency	☐	☐

12	Team size limits and permitted program combinations are set per course by the instructor. Teams that stop complying after a rule change are flagged for review, not removed. The values for the course are correct.	`FR_ValidateTeamRules`, course data in the data dictionary	☐	☐
13	The rules for who can read each type of record (proposals, projects, profiles, applications, approval feedback, and the TA's name) are correct, with nothing too open or too closed.	`DPR_VisibilityMatrix`, visibility tables	☐	☐
14	The items listed as not done (IP resolution, matching supervisors to student projects, meeting scheduling, direct messaging, post-approval feedback requests) are correct, and the priorities of the development phases match the instructor's biggest pain points.	Scope section, Development Plan	☐	☐

Other comments or corrections:

Reviewer: _____ Date: _____

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

7.2 How and Where Used

7.3 Verification of AI-Generated Content

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Ayush:

1. What went well while writing this deliverable?

   I think it was pretty easy to come up with this plan. We all have extensive experience with verification and validation from our course work and industry experience: unit testing, integration testing, E2E testing, and CI/CD and automation pipelines. We also know how important it is to validate with customers that we are building the right thing, and I have experience with regular demoing to a customer for exactly that reason. All of our past experience has allowed us to come up with a good plan for this deliverable.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Time management and prioritization with so much going on right now. The plan itself came together easily, but finding the time to actually sit down and write it up was hard with midterms and other assignments all landing around the same time. I resolved this by blocking out a few set times in the week for this deliverable and writing it in smaller chunks, instead of waiting for a big free afternoon that never showed up.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   For me, it is unit testing in Python with pytest and in React with Jest. I have experience with Vitest in Angular, but I have never unit tested a React project, so I expect the ideas to carry over but the tooling to feel new. As a team, we will also need Playwright for the end-to-end tests, integration and concurrency testing against a real PostgreSQL database (including stubbing Avenue and the email service), and setting up Locust load tests and mutation testing in GitHub Actions.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   For React unit testing, two approaches are reading the Jest and React Testing Library documentation, and building a small throwaway React app with a few component tests to get

hands-on practice. I will do both: the documentation first for the basics, then the small project, because I learn fastest by doing things. YouTube tutorials are a good backup when the documentation does not make something click. The same two approaches (documentation plus a small practice project) work for the other skills on the team list, and the rest of the team will pick whichever suits them for the tool they own.